

PRACTICAL 6

Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. one chopstick may be picked up by any one of its adjacent followers but not both.

Write a program to solve the problem using process synchronization

Code:

```
GNU nano 6.2 dining.c *
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t chopstick[N];

void* philosopher(void* arg)
{
    int id = *(int*)arg;

    while (1)
    {
        printf("Philosopher %d is THINKING\n", id);
        sleep(1);

        sem_wait(&chopstick[id]);
        sem_wait(&chopstick[(id + 1) % N]);

        printf("Philosopher %d is EATING\n", id);
        sleep(2);

        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id + 1) % N]);
    }
}

int main()
{
    pthread_t ph[N];
    int ids[N];
    int i;

    for (i = 0; i < N; i++)
    {
```

```
        sem_wait(&chopstick[id]);
        sem_wait(&chopstick[(id + 1) % N]);

        printf("Philosopher %d is EATING\n", id);
        sleep(2);

        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id + 1) % N]);
    }
}

int main()
{
    pthread_t ph[N];
    int ids[N];
    int i;

    for (i = 0; i < N; i++)
    {
        sem_init(&chopstick[i], 0, 1);
    }

    for (i = 0; i < N; i++)
    {
        ids[i] = i;
        pthread_create(&ph[i], NULL, philosopher, &ids[i]);
    }

    for (i = 0; i < N; i++)
    {
        pthread_join(ph[i], NULL);
    }

    return 0;
}
```

```
tanuja@Tanuja: ~  
tanuja@Tanuja:~$ nano dining.c  
tanuja@Tanuja:~$ gcc dining.c -lpthread  
tanuja@Tanuja:~$ ./a.out
```

Output :

```
tanuja@Tanuja:~$ nano dining.c  
tanuja@Tanuja:~$ gcc dining.c -lpthread  
tanuja@Tanuja:~$ ./a.out  
Philosopher 0 is THINKING  
Philosopher 1 is THINKING  
Philosopher 2 is THINKING  
Philosopher 3 is THINKING  
Philosopher 4 is THINKING  
Philosopher 0 is EATING  
Philosopher 2 is EATING  
Philosopher 2 is THINKING  
Philosopher 0 is THINKING  
Philosopher 4 is EATING  
Philosopher 1 is EATING  
Philosopher 1 is THINKING  
Philosopher 4 is THINKING  
Philosopher 3 is EATING  
Philosopher 0 is EATING  
Philosopher 3 is THINKING  
Philosopher 2 is EATING  
Philosopher 4 is EATING  
Philosopher 0 is THINKING  
Philosopher 4 is THINKING  
Philosopher 2 is THINKING  
Philosopher 1 is EATING  
Philosopher 3 is EATING  
Philosopher 0 is EATING  
Philosopher 2 is EATING  
Philosopher 3 is THINKING  
Philosopher 1 is THINKING  
Philosopher 2 is THINKING  
Philosopher 1 is EATING  
Philosopher 4 is EATING  
Philosopher 0 is THINKING  
^C  
tanuja@Tanuja:~$ |
```